

播客笔记

陈天奇：机器学习系统，长期主义，初心， XGBoost，MXNet，TVM，MLC LLM， OctoML | WhynotTV Podcast #3

Codex & 五道口纳什

2026 年 7 月 19 日



视频作者/频道：WhynotTV Podcast

发布日期：2025

视频时长：2 小时 40 分钟

视频链接：<https://www.bilibili.com/video/BV1s6pgzLE3y/>

目录

1 早年经历与成长	2
1.1 童年与计算机的初遇	2
1.2 ACM 班与系统思维	2
1.3 本章小结	2
2 XGBoost: 树模型的极致	2
2.1 诞生的背景	2
2.2 核心设计哲学	3
2.3 Kaggle 时代的统治力	3
2.4 本章小结	3
3 MXNet: 深度学习框架之争	4
3.1 从 XGBoost 到 MXNet	4
3.2 Python-First 的先见之明	4
3.3 框架战争的教训	4
3.4 本章小结	5
4 TVM: 深度学习编译的开创	5
4.1 问题起源	5
4.2 计算与调度分离	5
4.3 VTA: 可编程 NPU 的前瞻设计	6
4.4 本章小结	6
5 MLC LLM: 让大模型无处不在	6
5.1 从 TVM 到 MLC	6
5.2 Personal Computer 的类比	7
5.3 技术挑战与路线	7
5.4 本章小结	7
6 机器学习系统: 作为一个学科	7
6.1 为什么要定义”机器学习系统”	7
6.2 从”调参”到”系统工程”	8
6.3 边缘 AI 的想象空间	8
6.4 本章小结	9
7 OctoML: 创业的试炼	9
7.1 从 Apache TVM 到 OctoML	9
7.2 创业 vs 科研	9
7.3 本章小结	9
8 CMU 教职与指导哲学	9
8.1 为什么选择教职	9
8.2 Carlos Guestrin 的影响	10

8.3	作为导师的风格	10
8.4	本章小结	10
9	长期主义与价值观	10
9.1	被问到”想对十年前的自己说什么”时的沉默	10
9.2	为什么”现在保持初心更难”	11
9.3	给年轻研究者的建议	11
9.4	本章小结	11
10	总结与延伸	12
10.1	技术路线的演进逻辑	12
10.2	核心主张	12
10.3	开放问题	12
10.4	拓展阅读	13

1 早年经历与成长

1.1 童年与计算机的初遇

陈天奇的计算机启蒙始于少年时期。他在高二时自学编写编译器——这个经历对他后来的技术路线产生了深远影响。编译器是连接高级语言与底层硬件的桥梁，这种“将一种形式翻译为另一种形式”的思维方式，后来贯穿了他从 XGBoost 到 TVM 的整个技术生涯。

编译器的思维方式

编译器将一个领域特定语言 (DSL) 的表示翻译为另一个更底层、更可执行的表示。这种“翻译层”的思想，后来在 TVM 中被系统化为“计算与调度分离” (decouple compute from schedule) 的核心设计原则——将算法描述 (compute) 编译为特定硬件的执行方案 (schedule)。

他在上海交通大学 ACM 班打下系统功底。ACM 班的训练强调算法竞赛与系统编程的结合，这为他日后做高性能机器学习系统提供了稀缺的能力组合：既懂算法，又懂底层系统。

1.2 ACM 班与系统思维

ACM 班的经历塑造了陈天奇的系统思维。他提到 ACM 班的导师余荣老师强调“低调做人，高调做事”——这一价值观在他后续的科研生涯中反复体现。与很多追逐热点的研究者不同，陈天奇始终选择“重要但冷门”的方向深耕。

系统思维与长期主义

陈天奇反复强调一个核心观点：**做平庸的问题和做重要的问题花费的时间几乎一样**。因此，选择的问题本身就决定了你能达到的上限。这种“问题选择”的意识，是他从大学阶段就开始养成的思维习惯。

1.3 本章小结

陈天奇的早年经历揭示了两个关键线索：编译器思维（从高二自学编译器开始）和系统思维（从 ACM 班训练开始）。这两条线索在他后续的所有项目中都有体现——无论是 XGBoost 对树模型计算图的高效实现，还是 TVM 对深度学习算子的编译优化。

2 XGBoost：树模型的极致

2.1 诞生的背景

XGBoost (eXtreme Gradient Boosting) 诞生于陈天奇在华盛顿大学 (UW) 攻读博士期间。当时 Kaggle 竞赛正兴，梯度提升树 (Gradient Boosting Tree) 在结构化数据上表现优异，但现有的实现（如 R 语言的 gbm 包、scikit-learn 的 GBDT）在性能和可扩展性上存在明显短板。

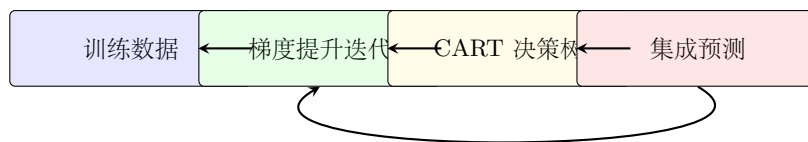


图 1: XGBoost 的核心训练流程：每轮迭代拟合前一轮的残差，所有树的预测加权求和

2.2 核心设计哲学

陈天奇对 XGBoost 的设计哲学可以概括为三个维度：

1. **模型的容量不应受限**：树模型的一个关键特性是”可任意增长”（arbitrary growth）——随着数据量增加，树的深度和数量都可以相应扩展。这与固定参数数量的线性模型形成鲜明对比。
2. **系统级优化**：XGBoost 不只是算法实现，更是系统工程。它引入了列块（column block）数据结构来加速分裂点搜索、利用缓存感知（cache-aware）的内存访问模式、支持分布式训练和核外计算（out-of-core）。
3. **可扩展的正则化**：在传统 GBDT 的目标函数中显式加入正则化项（树的叶子节点权重和叶子数量），使其比传统方法更不容易过拟合。

XGBoost 目标函数

XGBoost 在第 t 轮迭代时的目标函数为：

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

其中 $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$ 是正则化项。

- l ：损失函数（如平方误差、对数损失）
- $\hat{y}_i^{(t-1)}$ ：前 $t-1$ 轮的累积预测
- f_t ：第 t 棵树的输出
- T ：叶子节点数， w_j ：第 j 个叶子的权重
- γ 、 λ ：正则化超参数

2.3 Kaggle 时代的统治力

陈天奇回忆，XGBoost 在 2015 年前后的 Kaggle 竞赛中几乎成为”标配”工具。29 个 Kaggle 冠军解决方案中，有 17 个使用了 XGBoost。但陈天奇对此保持清醒——他提到，”如果我们继续去优化 ResNet，最后会输得很惨”。这一判断直接推动了 MXNet 的诞生。

2.4 本章小结

XGBoost 的成功证明了陈天奇的一个核心信念：**系统工程的深度决定了算法能走多远**。XGBoost 不仅是算法的胜利，更是系统工程的胜利——高效的数据结构、精细的内存管理、可扩展的分布式设计，这些才是它在 Kaggle 中碾压对手的真正原因。

3 MXNet: 深度学习框架之争

3.1 从 XGBoost 到 MXNet

陈天奇回忆, MXNet 的起点是对”下一步该做什么”的思考。在 XGBoost 获得成功后, 他意识到树模型在非结构化数据(图像、语音、文本)上的能力有限, 深度学习才是未来的大方向。

MXNet 的早期团队成员包括李沐 (Mu Li)、Alex Smola 等, 他们在 UW 和 CMU 联合推进。陈天奇描述当时的氛围: ”我们刚开始做 MXNet 的时候, 完全不会担心 TensorFlow 能对我们产生任何威胁。”

Python / Scala / R / Julia / C++ 多语言前端
符号式 (Symbolic) + 命令式 (Imperative) 混合编程
依赖引擎 (Dependency Engine) — 自动并行与调度
后端适配层: CPU / GPU / 分布式

图 2: MXNet 架构概览: 混合式编程模型 + 依赖引擎 + 多后端适配

3.2 Python-First 的先见之明

在 MXNet 诞生的年代 (2014–2015 年), 主流的深度学习框架生态是:

- Caffe: C++ 核心, Python 绑定, 但以配置文件 (prototxt) 驱动为主
- Torch7: Lua 语言, 生态较小
- Theano: Python, 但编译慢, 调试困难
- TensorFlow: 刚发布, 静态图为主

陈天奇团队做了一个在当时看来不显然的决定: **Python First**。他们同时支持符号式编程和命令式编程 (类似今天的 PyTorch eager mode), 这一设计后来被证明是正确的方向。

符号式 vs 命令式编程

- **符号式 (Symbolic)**: 先定义计算图, 后执行。优势是图优化、内存复用、跨设备部署。典型代表: TensorFlow 1.x、Theano。
- **命令式 (Imperative)**: 逐行执行, 即写即得。优势是调试方便、控制流自然。典型代表: PyTorch、NumPy。
- MXNet 的创新: **同时提供两种接口**, 用户可以在同一个模型中混用——这是后来 JAX 等框架也在探索的方向。

3.3 框架战争的教训

当被问到对框架战争的看法时, 陈天奇的反思格外坦诚。他认为 MXNet 输给 PyTorch/TensorFlow 的核心原因是:

1. **生态建设不足**: 模型 zoo、预训练权重、教程社区的建设比对手晚了太多

2. **团队资源差距**: Google (TensorFlow) 和 Facebook (PyTorch) 可以投入数倍甚至数十倍的工程资源
3. **时机与路径依赖**: 当学术界/工业界已经开始使用某个框架, 切换成本极高

框架战争的深层教训

陈天奇指出一个容易忽略的事实: 框架的胜负不取决于技术优劣。MXNet 在技术架构上有很多先进设计 (混合编程、依赖引擎、轻量级), 但”技术先进”不是框架取胜的充分条件。生态、社区、资源投入、时机, 这些因素在系统工程中往往比技术本身更重要。

3.4 本章小结

MXNet 的经历让陈天奇更深刻地理解了”现实”——如何通过现实的方式去实现理想。他坦言这段经历让他的做事方式变得更加务实。从 MXNet 的教训中, 他提炼出了 TVM 的方向: 不再做框架, 而是做框架之下的编译基础设施。

4 TVM: 深度学习编译的开创

4.1 问题起源

MXNet 之后, 陈天奇开始思考一个更深层的问题: **如何让深度学习模型高效运行在各种不同的硬件上?** 这个问题在当时几乎无人问津——因为当时的共识是”找几家大厂的工程师手工为每个硬件写 kernel 就好了”。

但陈天奇从编译器的视角看到了一种不同的可能性: 深度学习算子本质上是一种计算模式 (compute pattern), 如果能将计算描述与硬件调度分离, 就可以自动生成针对不同硬件的优化代码。

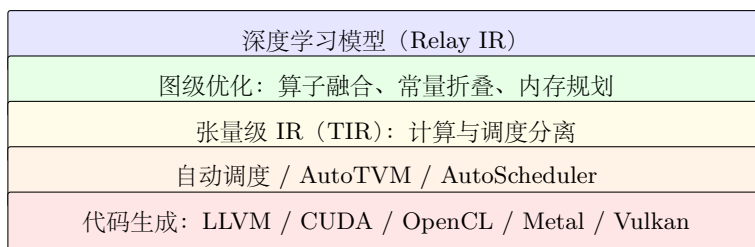


图 3: TVM 编译栈: 从模型到硬件指令的多层编译流水线

4.2 计算与调度分离

TVM 最核心的设计理念是 **Decouple Compute from Schedule**:

- **Compute**: 描述”做什么”——数学运算本身, 与硬件无关, 如矩阵乘法的数学定义
- **Schedule**: 描述”怎么做”——在特定硬件上的优化策略, 如 tiling、vectorization、parallelization、cache hierarchy

TVM 核心抽象

一个 TVM 算子由两部分组成：

算子 = Compute（计算描述） + Schedule（调度策略）

- Compute 是平台无关的数学定义，只需写一次
- Schedule 是平台相关的优化方案，通过自动搜索（AutoTVM）或手写模板生成
- 同一套 Compute 可以通过不同的 Schedule 生成针对 CPU、GPU、NPU 等不同后端的代码

4.3 VTA：可编程 NPU 的前瞻设计

TVM 的一个子项目 VTA（Versatile Tensor Accelerator）展示了对 NPU 架构的前瞻思考。陈天奇指出，当时大多数 NPU 是”完全硬连线的”（fully hardwired）——不可编程，只能做固定功能。VTA 从编译器的角度出发，设计了一套**可编程的微指令集**，使得同一个加速器可以执行多种深度学习算子。

VTA 的设计洞察

VTA 的关键洞察：将矩阵运算分解为更小的步骤（小矩阵乘法和内存读写），用一套统一的指令集编程，通过指令重叠（overlap）实现高效执行。这种设计使得 NPU 不再是固定的硬件功能块，而是一个可编程的张量处理器——在 ASIC 的效率和通用处理器的灵活性之间找到了平衡点。

4.4 本章小结

TVM 代表了陈天奇从”做框架”到”做编译基础设施”的战略转型。与其在框架层面与 Google/Facebook 竞争，不如在更底层的编译优化层面建立自己的壁垒。这种”向下沉”的策略后来被证明非常成功——TVM 成为 Apache 顶级项目，被亚马逊、华为、高通等公司广泛采用。

5 MLC LLM：让大模型无处不在

5.1 从 TVM 到 MLC

MLC（Machine Learning Compilation）LLM 是 TVM 思路在 LLM 时代的延续和升级。陈天奇回忆，在 2022 年底 ChatGPT 发布后，他很快意识到一个新的问题：**如何让大语言模型运行在每个人的设备上，而不只依赖云端 API？**

模型格式: HuggingFace / PyTorch / GGUF / ...
量化压缩: int4 / int3 / 混合精度
MLC 编译: TVM Unity IR → 平台代码
运行时: CPU / GPU / Metal / Vulkan / WebGPU
部署目标: iPhone / Android / 笔记本 / 浏览器

图 4: MLC LLM 流水线: 从预训练模型到全平台部署

5.2 Personal Computer 的类比

陈天奇用了一个精妙的类比来解释 MLC LLM 的愿景:

Mainframe → Personal Computer 的历史类比

历史上, 计算机经历了从大型机 (Mainframe) 到个人电脑 (Personal Computer) 的转变。今天的大语言模型正处在一个类似的转折点:

- **Mainframe 时代:** 所有计算集中在云端, 用户通过终端访问。对应今天的 ChatGPT API 模式。
- **PC 时代:** 每个人有自己的计算设备, 本地运行软件。对应未来每个人设备上运行本地 LLM。
- **关键前提:** 本地 LLM 的能力需要足够强。MLC 的工作就是让这个”足够强”更快到来。

5.3 技术挑战与路线

MLC LLM 面临的核心技术挑战:

1. **量化:** 大模型参数量巨大 (7B-70B+), 需要通过 int4/int3 量化在精度和速度之间找到平衡
2. **跨平台:** iPhone (Metal)、Android (OpenCL/Vulkan)、笔记本 (CUDA/CPU)、浏览器 (WebGPU) 各平台差异巨大
3. **内存受限:** 移动设备内存通常只有 4-8GB, 需要精细的内存规划和 KV-cache 管理

5.4 本章小结

MLC LLM 将 TVM 的编译思想从”通用深度学习模型”聚焦到了”大语言模型”, 并且在目标平台上从”云端/服务器”扩展到”个人设备/浏览器”。陈天奇将 MLC LLM 定位为”让机器学习系统工程像写神经网络一样简单”——这是他从 XGBoost 时代延续至今的愿景。

6 机器学习系统: 作为一个学科

6.1 为什么要定义”机器学习系统”

在访谈中, 陈天奇花了相当长的篇幅阐述他对”机器学习系统” (Machine Learning Systems) 作为一个学科的看法。他指出, 机器学习的历史通常被讲述为”算法进步史” (AlexNet → ResNet →

Transformer → GPT)，但系统层面的进步同样关键，甚至更关键——没有高效的训练和推理系统，这些算法的实际价值将大打折扣。

机器学习系统的定义

机器学习系统是研究如何高效实现、训练、部署机器学习模型的学科，涵盖：

- **训练系统**：分布式训练、混合精度、梯度压缩、流水线并行
- **推理系统**：模型量化、算子融合、内存优化、批处理调度
- **编译系统**：计算图优化、算子代码生成、硬件后端适配
- **部署系统**：模型服务、边缘端推理、跨平台适配

6.2 从”调参”到”系统工程”

陈天奇观察到，当前 ML 系统工程的一个瓶颈是门槛仍然很高。”你写一个神经网络已经非常简单了，在 Python 里面写几行就可以做到。但做机器学习系统工程开发还远远没有那么简单。”他希望能让 ML 系统工程也变得像写 PyTorch 代码一样简单——这正是 MLC 项目的使命。

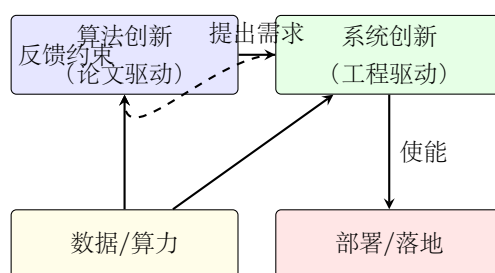


图 5: 算法与系统的共生关系：系统使能算法，算法提出系统需求，两者在约束中互相推动

6.3 边缘 AI 的想象空间

陈天奇对边缘 AI (Edge AI) 的前景非常乐观。他将当前的 LLM 生态类比为计算机发展史上的 Mainframe 时代——所有智能集中在云端数据中心。但正如 Personal Computer 最终改变了计算的面貌，他相信**本地 LLM** 也会改变 AI 的使用方式。

为什么需要本地 LLM

- **隐私**：敏感数据（个人笔记、医疗记录、金融信息）不应离开用户设备
- **延迟**：云端推理的网络延迟（100–500ms）对实时交互（如语音助手）是致命问题
- **离线可用**：飞机上、地下室、野外——网络不是随时可用的
- **成本**：云端 GPU 推理成本高昂，本地推理边际成本接近零
- **个性化**：本地模型可以与用户的个人数据深度绑定，提供定制化体验

6.4 本章小结

”机器学习系统”的学科界定是陈天奇对领域的一个重要贡献。他不仅做了 XGBoost、MXNet、TVM、MLC 这几个具体项目，更通过这些项目逐步定义和充实了”机器学习系统”这个学科的内涵——从”调参工具”到”系统工程”，从”少数人的手艺”到”可复用的基础设施”。

7 OctoML：创业的试炼

7.1 从 Apache TVM 到 OctoML

OctoML 是陈天奇与 TVM 核心团队在 2019 年创立的公司，目标是将 TVM 技术商业化。陈天奇坦率地分享了这段经历的种种挑战和认知转变。

OctoML 商业模式的三次迭代

- 阶段一（2019–2021）：**做 TVM 的企业级工具/平台，目标客户是硬件厂商和 ML 部署团队。
问题：市场太小，TVM 的开源版本已经足够好。
- 阶段二（2022–2023）：**转型做 ML 部署 SaaS 服务——类似 Fireworks、Together AI 的商业模式，帮客户部署和优化模型。
问题：竞争激烈，大模型 API 服务的差异化不够明显。
- 阶段三（2023–至今）：**聚焦特定垂直领域的端到端优化方案。
方向：让”部署”这件事本身变得更简单、更标准化。

7.2 创业 vs 科研

陈天奇分享了创业与科研的深刻比较：

- **科研：**追求”从 0 到 1”的突破，失败是可接受的结果，探索本身就有价值
- **创业：**追求”Product-Market Fit”，必须找到真正的付费客户，商业逻辑比技术先进性更重要
- **互补：**创业让他”更现实”——知道如何通过现实的方式去实现理想

7.3 本章小结

OctoML 的经历让陈天奇从纯粹的研究者变成了一个更全面的 builder。他形容这段经历”让我更现实，我会知道怎么样能够通过现实的方式去更好地实现理想”。这种”理想主义 + 现实主义”的双重思维，是他与其他纯学术研究者最显著的区别之一。

8 CMU 教职与指导哲学

8.1 为什么选择教职

陈天奇在博士毕业时就决定走教职路线。他的理由有三个：

1. **喜欢科研的不确定性：**做没有人知道答案的问题，这种探索感是不可替代的

2. 喜欢和同学们一起创造：他提到”和同学们一起去解决新的东西非常有趣”
3. 喜欢教学：把自己的知识和经验传递下去，帮助下一代研究者成长

8.2 Carlos Guestrin 的影响

陈天奇多次提到博士导师 Carlos Guestrin 对他的深远影响。Guestrin 不仅是 XGBoost 项目的共同推动者，更在科研方法论上给陈天奇留下了深刻的印记：

Guestrin 的科研方法论

- 做有影响力的问题：不要满足于增量改进，要寻找”能改变游戏规则”的问题
- 论文要 **accessible**：好的研究应该让读者容易理解，而不是用复杂公式吓退别人
- 从 **Research** 到 **Adoption**：一个好的研究项目应该能够从 paper 阶段推动到真正被人使用的阶段

8.3 作为导师的风格

陈天奇描述自己的指导风格：延续 Guestrin 的很多理念，但同时加入了自己的特色。他强调：

- 团队合作：机器学习系统是一个需要团队协作的方向，单打独斗很难走远
- 做生的、做有影响力的工作：鼓励学生选择”难但重要”的问题，而不是”容易发论文”的问题
- 跨领域视野：希望学生能够同时理解算法、系统、硬件三个层面

8.4 本章小结

陈天奇的教职身份（CMU Assistant Professor）与他的创业者身份（OctoML co-founder）形成了一种有趣的张力。他自己描述为”80% 教授，20% 创业者”——这种双重身份让他能够在学术深度和产业落地之间找到平衡。

9 长期主义与价值观

9.1 被问到”想对十年前的自己说什么”时的沉默

这期播客最打动人心的片段之一，是主持人对陈天奇的一个提问：”你会对十年前、二十年前的陈天奇说些什么？”陈天奇沉默了近 20 秒后回答：

”可能要反过来——我需要过去的我对现在的我，现在的我对未来的我说：记住自己对自己的承诺，坚持自己的理想，往下走下去。”

这个回答揭示了陈天奇价值观的核心：不是”现在的我”去纠正”过去的我”，而是”过去的初心”来提醒”现在的我”不要迷失。

长期主义的实践

陈天奇对长期主义的理解不是抽象的口号，而是非常具体的操作原则：

1. **问题选择**：做重要的问题和平庸的问题花费的时间几乎一样——选择本身就决定了上限
2. **接受失败**：做难的问题必然意味着不确定性和失败，关键在于“be willing to take the risk”
3. **Reinvent yourself**：从 XGBoost 到 MXNet 到 TVM 到 MLC，每一次都是一次自我革新
4. **勇气来自过去的自己**：当你犹豫不决时，回想自己最初为什么出发——那份初心会给你继续的勇气

9.2 为什么“现在保持初心更难”

当陈天奇说“现在保持初心更难”时，他提到了一个悖论：

- **资源多了**：现在的陈天奇能调动的资源是十年前的“不是一个数量级”
- **选择多了**：但选择越多，反而越容易迷失——“你会开始担心”
- **对比效应**：当你看到别人走得更快、做得更“热”时，会动摇自己对冷门方向的坚持

选择的悖论

陈天奇指出一个常见的陷阱：当资源充足时，反而更容易做平庸的决策——因为“安全”的路径看起来更加诱人。他说：“我们刚开始做 MXNet 的时候，完全不会担心 TensorFlow 能对我们产生什么威胁。”那种“一无所有所以无所畏惧”的状态，反而是创新的最佳土壤。

9.3 给年轻研究者的建议

陈天奇对年轻研究者（特别是 PhD 学生）的建议可以浓缩为三条：

1. **选择自己真正有 passion 的问题**：不要跟风，不要为了发论文而选择“安全”的方向
2. **要有系统级的视野**：既能看到具体的技术问题，也能理解问题在整个系统中的位置
3. **不要害怕失败**：他引用自己早期的失败经历——手搓 RBM 做 ImageNet classification 失败，ACM 竞赛失败——来说明失败本身就是科研的常态

9.4 本章小结

陈天奇的价值观可以概括为：**长期主义 × 敢失败 × 系统思维**。这是他在 20 年机器学习系统工程师生涯中提炼出的底层哲学，也是他所有技术决策的最终依据。

10 总结与延伸

10.1 技术路线的演进逻辑

回顾陈天奇 20 年的技术路线，可以看到一条清晰的演进逻辑：

XGBoost（树模型极致） → MXNet（框架层） → TVM（编译层） → MLC LLM（全栈部署）

这条路线中有几个关键的战略转折：

1. **从树模型到深度学习**（2014–2015）：意识到树模型的上限，主动转向深度学习框架
2. **从框架到编译**（2016–2017）：MXNet 在框架战中落败后，”向下沉”到编译基础设施
3. **从云端到边缘**（2022–至今）：MLC LLM 将焦点从服务器端推理转向个人设备部署

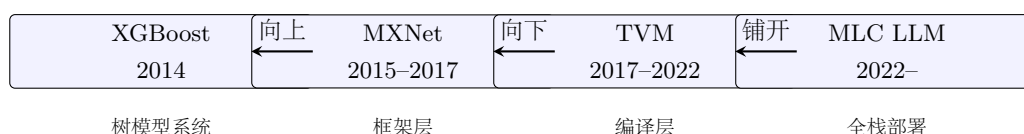


图 6: 陈天奇 20 年技术路线的演进：向上（框架）→ 向下（编译）→ 铺开（全平台）

10.2 核心主张

从 2 小时 40 分钟的深度访谈中，可以蒸馏出陈天奇的几个核心主张：

1. **系统价值被低估**：ML 的进步不只是算法的故事，系统工程的深度同样决定了模型能走多远。XGBoost 证明了这一点，TVM 也证明了这一点。
2. **选择本身就是最重要的决策**：做平庸的问题和做重要的问题花费的时间几乎一样。问题选择的眼界，来自长期积累的系统视野。
3. **在不确定性中找到确定性**：科研和创业都是高不确定性活动，”be willing to take the risk” 不是勇气问题，而是认知问题——理解失败是过程的一部分。
4. **不忘初心不是道德要求，是战略要求**：从过去的自己那里汲取勇气，不是因为”坚持”本身是美德，而是因为只有初心能告诉你什么是真正值得做的。
5. **现实是理想的实现方式**：OctoML 的创业经历教会陈天奇”通过现实的方式去更好地实现理想”——理想主义不排斥现实主义，相反，现实主义是理想主义的执行手段。

10.3 开放问题

陈天奇在访谈中留下了几个值得思考的开放问题：

- **本地 LLM 的能力何时能达到”足够好”的水平**？这决定了边缘 AI 的时间表
- **机器学习系统能否真正成为一个独立的学科**？目前它仍介于 CS Systems 和 ML 之间，边界模糊

- 当下一个范式转换来临时，什么会成为新的瓶颈？陈天奇的回答是”继续去解决让更少的人用更少的 engineering resource 去解决 ML 系统的问题”
- 如何在资源充裕时保持”敢失败”的心态？这是陈天奇自己也在思考的问题

10.4 拓展阅读

- 陈天奇个人主页: tqchen.com
- 陈天奇知乎文章「机器学习科研的十年」: zhuanlan.zhihu.com/p/74249758
- WhynotTV 播客主页: tairanhe.com/podcast_cn
- Apache TVM 项目: tvm.apache.org
- MLC LLM 项目: github.com/mlc-ai/mlc-llm